

AuraChess - Relazione di Progetto

Studenti: Riccardo De Medio, Federico Giorgetti, Nicolò Paganini, Michele Stanzani

Capitolo 1: Analisi

1.1 Descrizione e requisiti

AuraChess è un'applicazione desktop che riproduce il classico gioco degli scacchi, arricchito da funzionalità avanzate progettate per offrire un'esperienza interattiva e didattica. Il sistema permette di giocare una partita standard contro un'Intelligenza Artificiale (CPU) integrata, dotata di un motore di valutazione denominato "AuraEngine".

Una caratteristica distintiva del software è l'implementazione dell'**Aurometro**, un sistema che valuta in tempo reale la precisione delle mosse del giocatore umano rispetto alla mossa ottimale calcolata dalla CPU, restituendo un feedback visivo e testuale immediato.

Requisiti funzionali:

- **Gestione regole degli scacchi:** Il sistema deve validare e gestire i movimenti standard di tutti i pezzi, incluse le regole speciali: arrocco (corto e lungo), presa *en passant*, promozione del pedone, scacco e scacco matto.
- **Condizioni di patta:** Riconoscimento automatico di stallo, regola delle 50 mosse, triplice ripetizione e materiale insufficiente.
- **Motore scacchistico (CPU):** Un avversario artificiale in grado di valutare l'albero delle mosse possibili e prendere decisioni strategiche.
- **Sistema di salvataggio/caricamento:** Possibilità di salvare lo stato della partita su file locale e ricaricarlo per riprendere il gioco.
- **Gestione dello storico (Undo):** Possibilità di annullare le mosse giocate (rollback) sia per correggere errori che a scopo didattico.
- **Valutazione mosse e Report:** Calcolo della precisione della singola mossa e generazione di un report riepilogativo a fine partita.
- **Orologio di gioco:** Gestione del tempo tramite timer configurabile.

Requisiti non funzionali:

- **Reattività dell'interfaccia (UI):** L'interfaccia grafica deve rimanere fluida e responsiva anche durante i tempi di calcolo (pensiero) della CPU.
- **Disaccoppiamento:** Il codice deve mantenere una netta separazione tra logica di gioco, gestione dell'interfaccia e salvataggio dei dati.
- **Portabilità:** Il software deve essere eseguibile su diverse piattaforme desktop che supportino l'ambiente di esecuzione scelto.
- **Robustezza e Sicurezza:** Gestione sicura dei file di salvataggio (sanitizzazione dei nomi file) per prevenire accessi indesiderati al file system dell'utente.

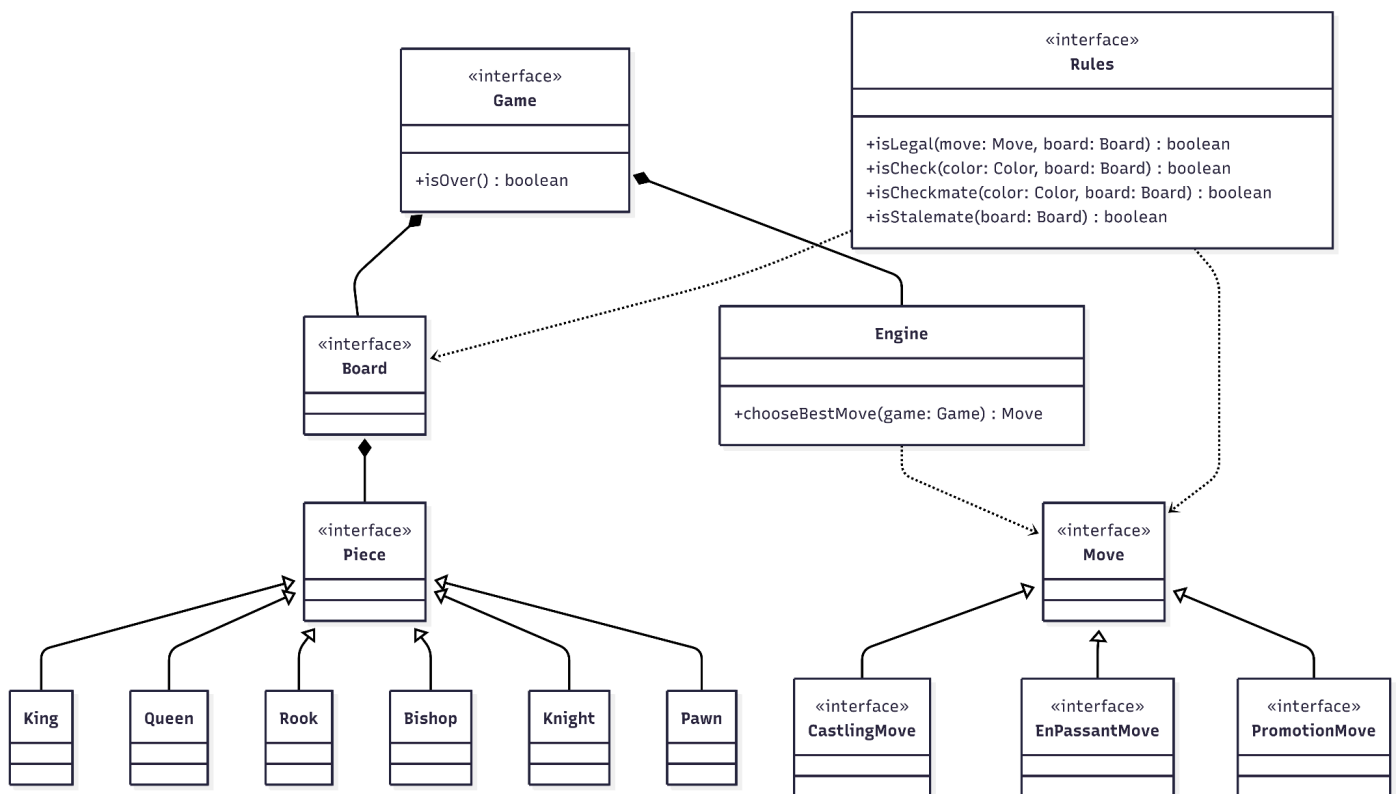
1.2 Modello del Dominio

AuraChess modella una sfida scacchistica tra un Giocatore umano e un motore di gioco automatizzato, identificato come Engine. Il sistema è orchestrato dall'interfaccia Game, che funge da supervisore dell'intera sessione di gioco, gestendo l'alternanza dei turni e lo stato di avanzamento della partita.

Il campo di gioco è rappresentato dall'entità Board, composta da un insieme definito di Position che ospitano i Piece. Ciascun pezzo è caratterizzato da un colore e da una posizione specifica sulla scacchiera; il modello prevede la specializzazione dei pezzi nelle classi concrete (King, Queen, Rook, Bishop, Knight, Pawn), ognuna soggetta a logiche di movimento differenziate.

La validità di ogni azione è delegata al modulo Rules, che monitora costantemente le configurazioni della Board per determinare se una Move sia conforme al regolamento. Rules è inoltre responsabile di valutare le condizioni di fine partita, verificando stati come lo scacco matto, lo stallo o la patta. Il sistema supporta inoltre mosse complesse, modellate attraverso la gerarchia di Move, che include sottoclassi specializzate per gestire le meccaniche di CastlingMove, PromotionMove ed EnPassantMove. Infine, l'entità Engine interagisce con il Game per analizzare lo stato corrente della scacchiera e determinare autonomamente la migliore mossa tattica da eseguire.

Schema UML:



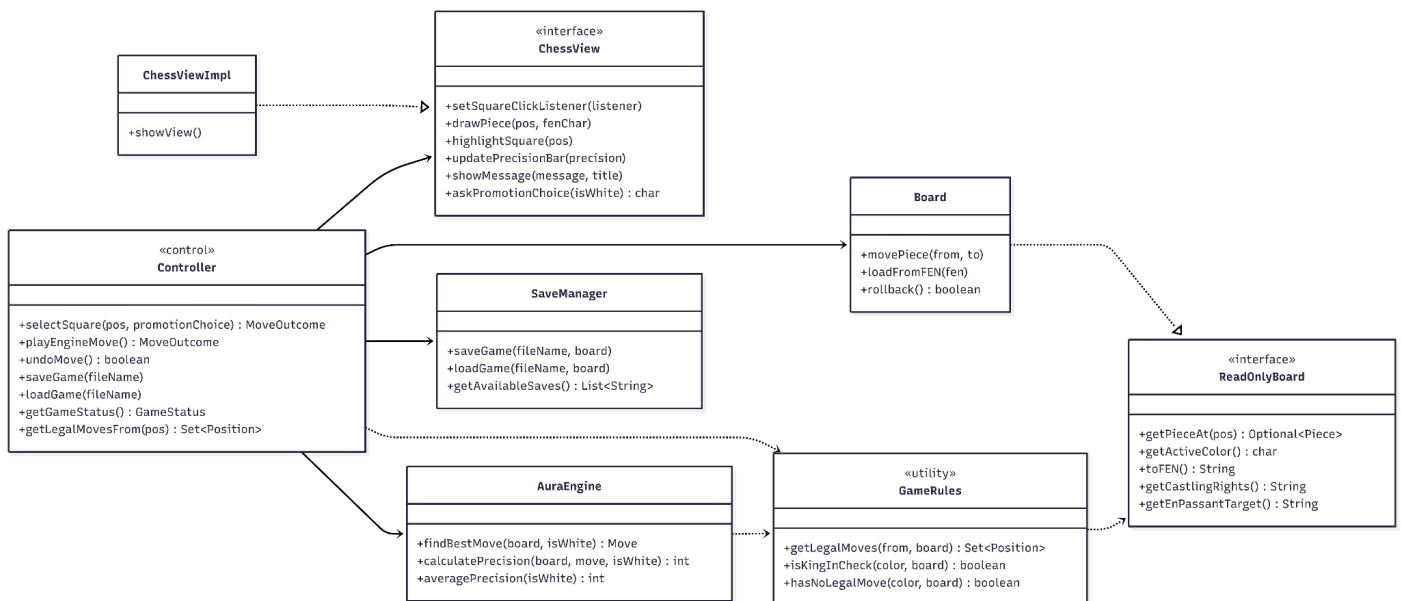
Capitolo 2: Design

2.1 Architettura

L'architettura di **AuraChess** si fonda sul pattern architetturale **MVC (Model-View-Controller)**, strutturato in modo efficace per la gestione dei turni, salvataggio, caricamento partite, tornare indietro di mosse e i commenti delle giocate.

1. **Model (scacchi.model.*)**: Contiene la logica di dominio pura. È completamente agnostico rispetto all'interfaccia grafica. Include l'intelligenza artificiale (**AuraEngine**), la rappresentazione della scacchiera (**BoardImpl**), le regole di gioco (**GameRules**), la gerarchia dei pezzi (**Piece**), il timer (**ChessClock**) e la persistenza (**SaveManagerImpl**).
2. **View (scacchi.view)**: L'interfaccia grafica implementata tramite Java Swing (**ChessViewImpl**). Riceve l'input dall'utente (click sulle celle, bottoni) e lo demanda al Controller. Si ridisegna passivamente interrogando il Controller o rispondendo ai suoi comandi.
3. **Controller (scacchi.controller.Controller)**: Gestisce il flusso della partita. Riceve gli eventi dalla View, aggiorna il Model, controlla le condizioni di vittoria/sconfitta e gestisce la sincronizzazione dei thread (es. calcolo mosse CPU in background).

Schema UML



Interazione e Flusso Dati

Dal Model alla View (Aggiornamento) Per garantire un forte disaccoppiamento, il Model non possiede alcun riferimento alla View. È il **Controller** che agisce da mediatore attivo: ad ogni mossa, annullamento (undo) o caricamento di un salvataggio, il Controller legge lo stato aggiornato dalla scacchiera (**BoardImpl**) e impartisce direttive mirate alla grafica. Invece di passare interi oggetti di dominio, il Controller utilizza metodi granulari esposti dall'interfaccia grafica (**comedrawPiece**, **highlightSquare** o **clearSquare**). Questo approccio garantisce che la View non riceva mai riferimenti mutabili del Model, impedendole di alterare inavvertitamente la logica di gioco e mantenendola un componente puramente passivo.

Dalla View al Controller (Input e Callback) L'interazione dell'utente segue un approccio rigorosamente guidato dagli eventi (Event-Driven) tramite l'uso di interfacce funzionali. L'interfaccia **ChessView** espone

metodi per la registrazione di *Listener*. All'avvio dell'applicazione, il Controller inietta i propri riferimenti di metodo in questi listener. In questo meccanismo di delega, quando l'utente clicca su una cella, la View si limita a notificare la coordinata, ignorando completamente le regole degli scacchi o l'esito di quell'input.

Gestione dei Turni e Asincronia Il Controller ricopre un ruolo cruciale nella gestione dell'eterogeneità dei turni (Giocatore Umano vs CPU). La strategia di attesa varia profondamente:

- **Turno Umano:** Il Controller attende passivamente gli eventi di input provenienti dalla View. Se l'utente tenta di interagire mentre l'avversario artificiale sta elaborando una mossa, l'input viene intenzionalmente ignorato grazie al lock logico garantito dal flag `volatile boolean engineThinking`, prevenendo stati inconsistenti come muovere un pezzo mentre la scacchiera è in fase di valutazione.
- **Turno CPU:** Quando spetta ad `AuraEngine` giocare, il Controller non blocca il flusso di esecuzione principale. Invocando il metodo `maybeTriggerEngineMove()`, il calcolo viene delegato a un thread di background tramite la classe `SwingWorker`. Ciò permette di scendere in profondità nell'albero delle mosse senza congelare l'interfaccia reattiva; a computazione terminata, il worker applica la mossa sull'Event Dispatch Thread (EDT) in totale sicurezza.

Sostituibilità della View Grazie all'astrazione garantita dall'interfaccia `ChessView`, il Controller ignora i dettagli implementativi della libreria grafica sottostante (Java Swing, racchiusa in `inChessViewImpl`). Sostituire l'interfaccia grafica, ad esempio migrando il progetto verso JavaFX, o implementando una UI testuale da riga di comando, impatterebbe esclusivamente l'implementazione visiva, lasciando totalmente inalterati la logica del Model e le direttive del Controller.

2.2 Design di dettaglio

Federico Giorgetti: Intelligenza Artificiale e Valutazione

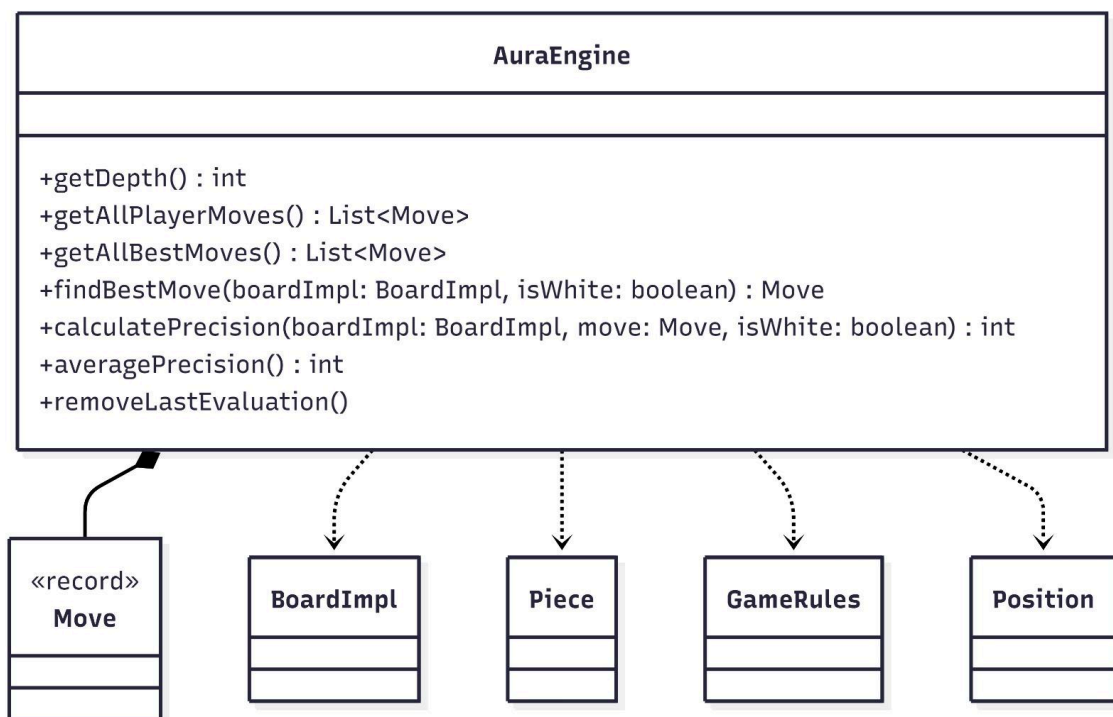
Motore Scacchistico e Ricerca della Mossa Migliore

- **Descrizione del problema:** Il requisito fondamentale per l'Intelligenza Artificiale era sviluppare un avversario CPU in grado di compiere scelte strategiche e tatticamente intelligenti in tempi di elaborazione ragionevoli, così da non compromettere la fluidità dell'esperienza di gioco. Negli scacchi, l'altissimo fattore di ramificazione causa un'esplosione combinatoria dell'albero delle mosse, rendendo impossibile un'analisi esaustiva in tempo reale.
- **Soluzione proposta (Algoritmo Minimax con Alpha-Beta Pruning):** La classe `AuraEngine` implementa una versione avanzata dell'algoritmo Minimax. Per mitigare la crescita esponenziale dei nodi, è stata integrata la tecnica di potatura *Alpha-Beta* (`minimaxingAlfaBetaPruning`). Questa ottimizzazione matematica permette all'algoritmo di ignorare (tagliare) interi rami dell'albero decisionale che si dimostrano peggiori rispetto a varianti già esplorate. Ciò riduce drasticamente il numero di nodi visitati (`nodesVisited`), permettendo al motore di raggiungere profondità di ricerca nettamente superiori a parità di tempo di calcolo.
- **Alternative scartate:** L'utilizzo dell'algoritmo Minimax puro (Standard Minimax). Questa strada è stata implementata inizialmente ma subito abbandonata per ragioni di prestazioni: senza la potatura dei rami sub-ottimali, i tempi di esecuzione crescevano in modo insostenibile già a profondità di calcolo superficiali, rendendo l'avversario artificiale inutilizzabile per una partita interattiva.

Valutazione Analitica dell'Utente ("Aurometro") e Sicurezza dello Stato

- **Descrizione del problema:** Il sistema doveva fornire un feedback didattico immediato (l'"Aurometro"), quantificando matematicamente la qualità della singola mossa appena giocata dall'utente. Parallelamente, era imperativo prevenire la corruzione dello stato della scacchiera durante i massicci cicli di andata e ritorno (move/undo) richiesti dalle simulazioni dell'engine.
- **Soluzione proposta (Calcolo della Perdita e Immutabilità tramite Record):** Per la valutazione, è stato ingegnerizzato il metodo `calculateLoss`. Il sistema calcola la valutazione della scacchiera risultante dalla mossa dell'utente e la confronta con la valutazione teorica che si sarebbe ottenuta giocando la *best move* individuata dall'engine. Questo differenziale (la "perdita" o "loss") viene poi normalizzato e trasformato in un indice di precisione percentuale (0-100%). Per blindare la sicurezza delle simulazioni, l'architettura interna della generazione delle mosse si appoggia nativamente ai costrutti `Record` di Java (`Move`, `PlacedPiece`, `UndoInfo`). Essendo intrinsecamente immutabili, questi DTO garantiscono che i parametri scambiati durante la profonda discesa ricorsiva dell'algoritmo non vengano mai accidentalmente alterati, evitando corruzioni catastrofiche dello stato del modello di dominio.
- **Alternative scartate:** Si era valutata l'idea di misurare la qualità della mossa dell'utente attraverso euristiche statiche superficiali (come il semplice conteggio dei pezzi catturati in quel turno). L'alternativa è stata scartata poiché non teneva conto delle implicazioni tattiche a lungo termine (ad esempio, una mossa che non cattura nulla ma previene uno scacco matto). Il calcolo differenziale sulla *best move* garantisce invece una misurazione oggettiva e profonda.

Schema UML



Michele Stanzani: Gerarchia e Movimento dei Pezzi

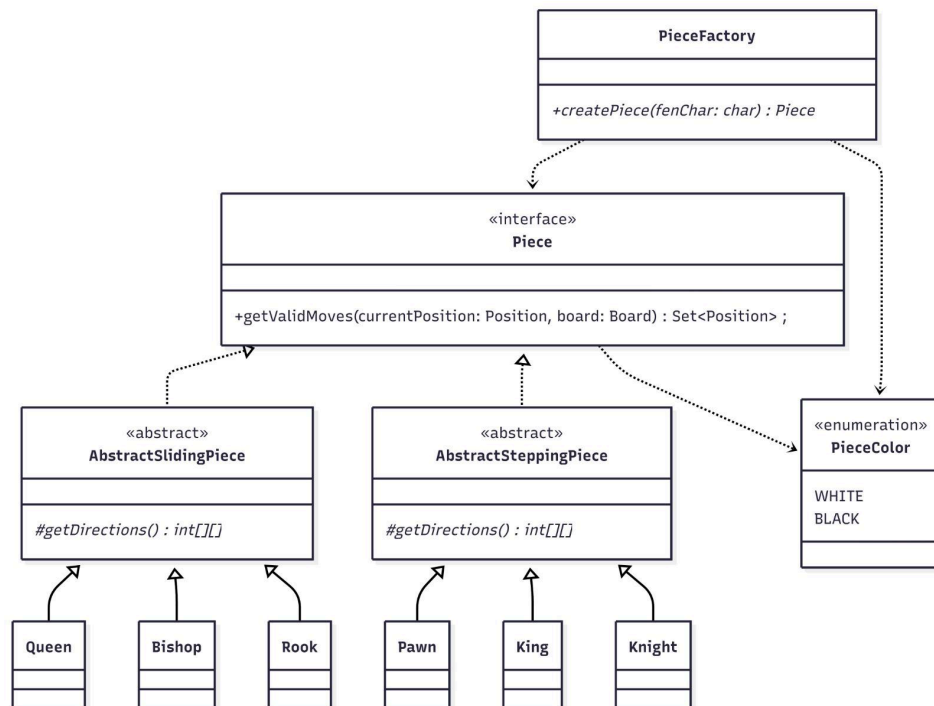
Modellazione Architeturale dei Movimenti

- **Descrizione del problema:** Negli scacchi, ogni pezzo possiede regole di movimento e di cattura strettamente peculiari. Centralizzare il calcolo di tutte le traiettorie possibili all'interno dell'entità **Board**, o peggio, all'interno di un'unica classe monolitica di validazione, avrebbe generato un codice estremamente rigido, fortemente accoppiato e di complessa manutenibilità. Un simile approccio avrebbe violato palesemente i principi cardine dell'ingegneria del software, in particolare il *Single Responsibility Principle* (SRP) e l'*Open/Closed Principle* (OCP).
- **Soluzione proposta (Template Method e Polimorfismo):** Per garantire la massima estensibilità e pulizia del codice, è stata progettata una gerarchia polimorfica basata sull'interfaccia **Piece**. Da questa astrazione principale diramano due classi astratte fondamentali: **AbstractSlidingPiece** (dedicata a Torre, Alfiere e Regina, che si muovono su linee continue) e **AbstractSteppingPiece** (per Re e Cavallo, che si muovono a salti discreti). Queste classi concretizzano il pattern **Template Method**: implementano il metodo **getValidMoves()** definendo lo scheletro dell'algoritmo di calcolo (scivolamento iterativo fino a collisione per i primi, salto singolo per i secondi), ma delegano la fornitura dei vettori direzionali specifici alle classi figlie concrete tramite il metodo astratto **getDirections()**. Il Pedone (**Pawn**), a causa della sua natura asimmetrica e complessa (movimento solo frontale, cattura solo diagonale, meccanica del doppio passo iniziale), bypassa le classi intermedie e implementa direttamente l'interfaccia **Piece**.

Istanziazione Sicura e Disaccoppiamento

- **Descrizione del problema:** Il caricamento e il ripristino dello stato della scacchiera avvengono tramite il parsing di stringhe in formato FEN. Senza un'adeguata separazione delle responsabilità, il modulo incaricato di decodificare la stringa FEN si sarebbe trovato costretto a conoscere intimamente l'intera gerarchia delle classi concrete dei pezzi per poterle istanziare, creando un pernicioso accoppiamento tra la logica di I/O (parsing) e le implementazioni fisiche del modello di dominio.
- **Soluzione proposta (Factory Method):** Per recidere questo accoppiamento, l'istanziamento dei pezzi a partire dai singoli caratteri FEN è stata delegata in toto alla classe **PieceFactory**, applicando il pattern **Factory Method**. Questa classe-fabbrica riceve il carattere identificativo (es. 'K' per il Re bianco, 'p' per il pedone nero) e restituisce l'istanza polimorfica già configurata correttamente. In questo modo, l'intrinseca complessità costruttiva dei vari pezzi viene incapsulata in un unico punto, mantenendo la **Board** e gli altri moduli dipendenti esclusivamente dall'interfaccia astratta **Piece**.
- **Alternative scartate:** L'istanziamento diretta tramite operatore **new** (**new Rook()**, **new Pawn()**) codificata staticamente all'interno del metodo di caricamento FEN della **BoardImpl**. Questa strada è stata abbandonata perché avrebbe legato in modo indissolubile il parser alle implementazioni specifiche, rendendo il codice fragile e dispersivo in vista di futuri refactoring della gerarchia dei pezzi.

Schema UML:



Riccardo De Medio: Controller e Regole Avanzate

Gestione delle Regole Avanzate e Simulazione dello Stato

- **Descrizione del problema:** La validazione delle regole complesse degli scacchi (scacco, scacco matto, presa en passant, restrizioni sull'arrocco) richiede di interrogare costantemente lo stato della scacchiera per prevedere scenari futuri. In particolare, per verificare se una mossa è legale, è necessario accertarsi che essa non lasci il proprio re sotto scacco. Eseguire questa verifica modificando direttamente l'oggetto **Board** principale esponeva il sistema al rischio di alterare in modo permanente e accidentale lo stato della partita corrente.
- **Soluzione proposta (Pure Fabrication e Decorator):** Tutta la logica di validazione delle mosse è stata scorporata ed estratta nella classe di utilità **GameRules**. Questa entità rappresenta un'implementazione da manuale del pattern **Pure Fabrication**: una classe non strettamente legata al dominio fisico degli scacchi, ma ingegnerizzata per massimizzare la coesione e minimizzare l'accoppiamento. Per simulare in totale sicurezza le mosse (ad esempio, il controllo preventivo dello scacco), **GameRules** si avvale di un **Record** interno denominato **SimulatedBoard**. Questa struttura agisce come un leggerissimo pattern **Decorator**: avvolge la board reale intercettando le chiamate di lettura e restituendo uno stato ipotetico, senza mai mutare l'array di base dei pezzi in memoria.
- **Alternative scartate:** L'approccio iniziale prevedeva di eseguire materialmente la mossa sulla **Board** reale, verificare lo stato del re e successivamente invocare un "undo" per ripristinare la situazione. Questa tecnica (*make-check-unmake*) è stata categoricamente scartata poiché scatenava effetti collaterali indesiderati (come l'aggiornamento errato dello storico FEN e dei contatori di patta) e inquinava la purezza dell'architettura. Anche la clonazione profonda (*Deep Copy*) della board ad ogni controllo è stata abbandonata per il grave overhead prestazionale.

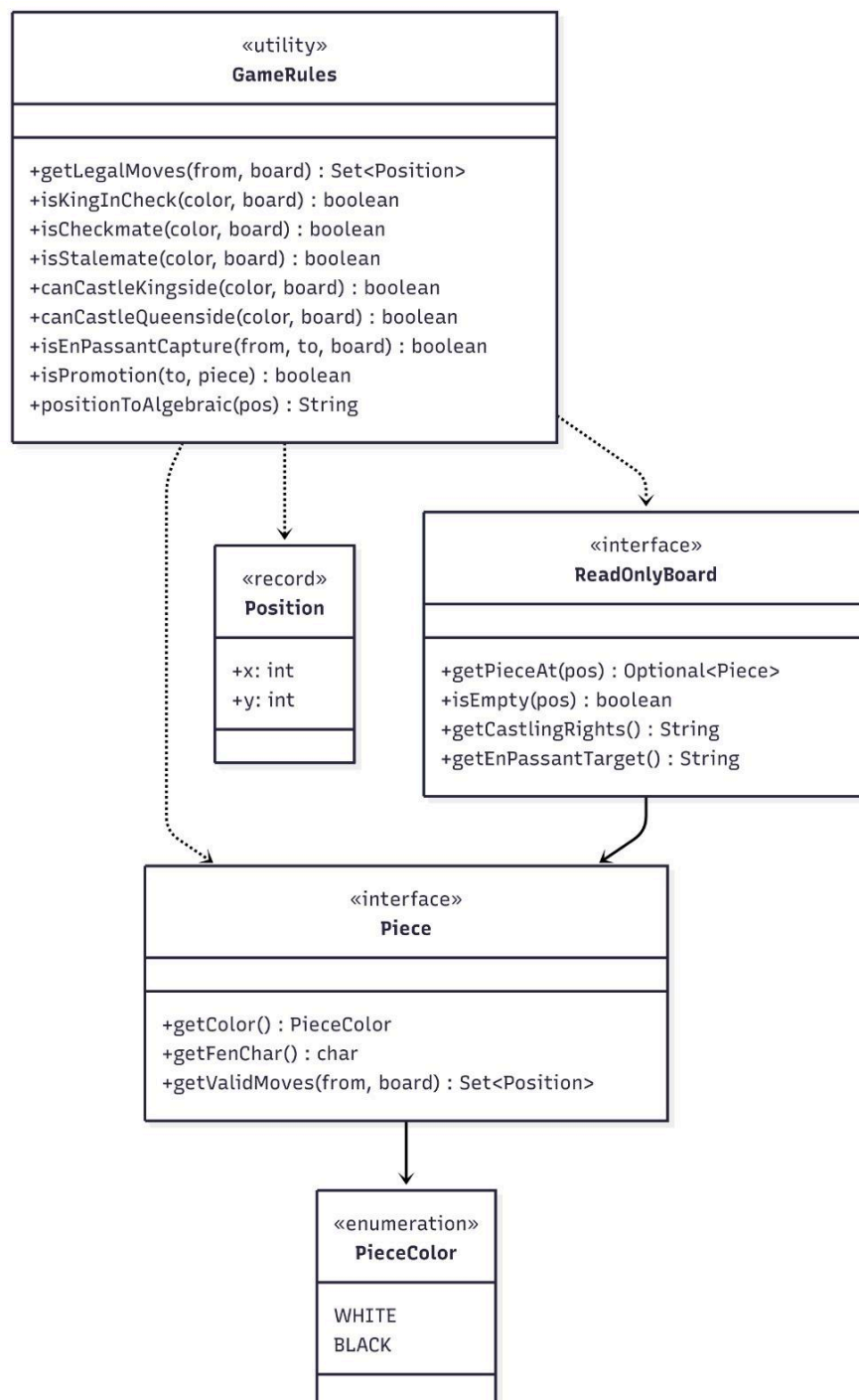
Selezione e l'uso degli Enum (Gestione Eventi)

- **Descrizione del problema:** La gestione dell'input utente per muovere un pezzo richiede un processo a due fasi (selezione della casella di partenza e selezione della destinazione). Gestire

questo flusso sequenziale attraverso semplici variabili booleane o blocchi **if-else** nidificati all'interno del Controller avrebbe generato "spaghetti code", rendendo difficile tracciare esiti complessi come mosse illegali, auto-scacco o deselezione.

- **Soluzione proposta (Macchina a stati leggeri e Enum):** Il flusso di interazione è stato incapsulato nel metodo **selectSquare**, che agisce come una macchina a stati basata sulla variabile **selectedSquare**. Per standardizzare e tipizzare le risposte di questo flusso, è stata creata l'enumerazione **MoveOutcome** (es. **SELECTED**, **DESELECTED**, **INVALID_SELECTION**, **MOVE_PLAYED**). In questo modo, l'esito di ogni interazione è semanticamente chiaro, facilitando il compito dell'interfaccia grafica che deve reagire di conseguenza (ad esempio, aggiornando i contorni delle caselle o avviando la CPU).
- **Alternative scartate:** Mantenere lo stato della selezione (quale casella è attualmente cliccata) all'interno della **ChessView**. Questa strada è stata scartata perché avrebbe violato il pattern MVC: la View deve essere "passiva" e non deve mantenere stati logici legati al dominio. Lo stato della selezione è a tutti gli effetti un dato logico di transizione che spetta al Controller.

Schema UML:



Nicolò Paganini

Board, Salvataggio e Storico (Rollback) e View

Gestione dello Stato, Storico e Salvataggio

- **Descrizione del problema:** Il sistema richiedeva di salvare lo stato della partita, mantenere uno storico completo per la funzione di "Undo" (annulla mossa) e permettere l'analisi di regole complesse come la triplice ripetizione o la regola delle 50 mosse. Salvare in memoria un intero oggetto `Board` (tramite clonazione profonda) ad ogni singola mossa avrebbe causato un overhead di memoria inaccettabile e rallentamenti nell'esecuzione.
- **Soluzione proposta (Notazione FEN e Pattern Memento):** Si è scelto di rappresentare l'intero stato della scacchiera (posizione dei pezzi, turno, diritti di arrocco, en passant e contatori) in una stringa compatta FEN (Forsyth-Edwards Notation). La classe `BoardImpl` mantiene una `Deque<String> history`. Ad ogni mossa, prima di spostare il pezzo, lo stato attuale viene serializzato in FEN e inserito nello stack, realizzando una reificazione efficiente del pattern **Memento**. Per eseguire il rollback ("Undo"), è sufficiente estrarre l'ultimo FEN e passarlo al metodo `loadFromFEN()`. Il `SaveManagerImpl` si occupa della persistenza scrivendo l'intero stack FEN (e i relativi timer) su un file `.fen` testuale all'interno di una directory di sistema nascosta (`.aurascacchi/saves`).
- **Sicurezza e Sanitizzazione:** Per prevenire vulnerabilità di sicurezza informatica (come il *Path Traversal*), è stata implementata un'espressione regolare (`UNSAFE_CHARS.matcher`) volta a sanitizzare i nomi dei file inseriti in input dall'utente, scartando i caratteri non sicuri.
- **Alternative scartate:** Si era valutato l'utilizzo della *Serializzazione nativa di Java* o il salvataggio di copie profonde (*Deep Copy*) dell'oggetto `Board`. Tali approcci sono stati scartati in quanto eccessivamente onerosi in termini di spazio e tempo di esecuzione, specialmente considerando la profondità dell'albero di gioco esplorato dall'IA.

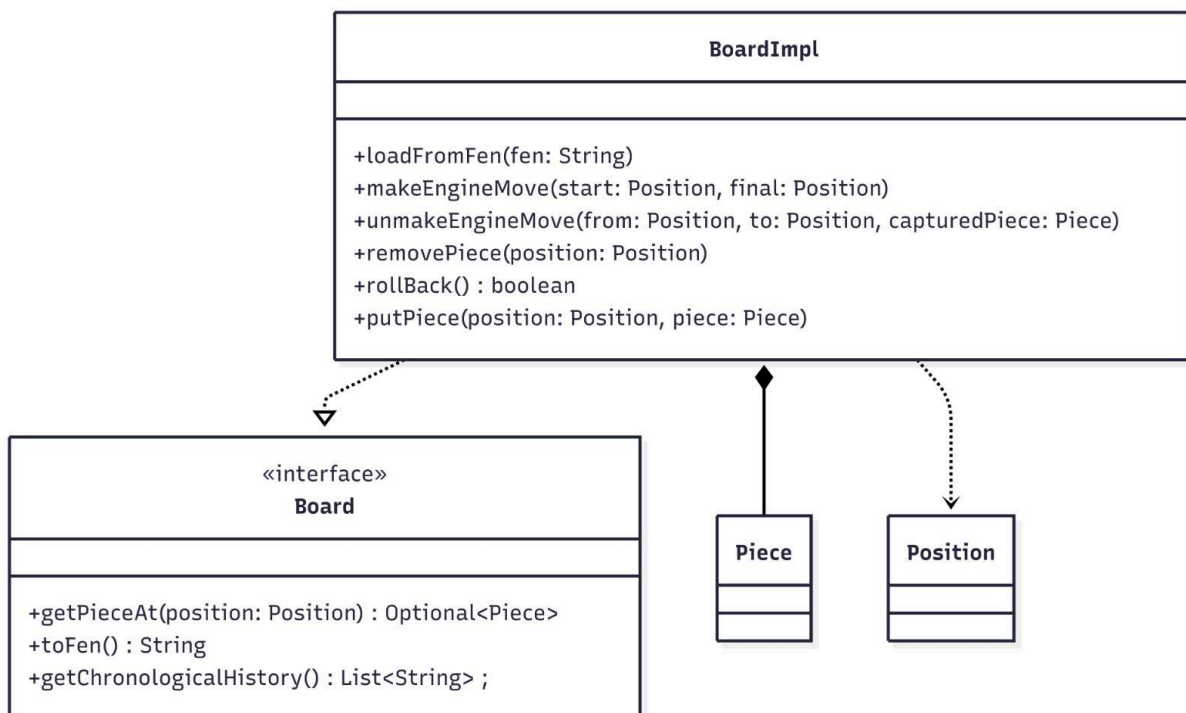
Ottimizzazione Strutturale per le Performance dell'AuraEngine

- **Descrizione del problema:** Inizialmente, la griglia di gioco era stata modellata utilizzando una `Map<Position, Piece>`. Tuttavia, l'*AuraEngine* (basato sull'algoritmo Minimax) necessita di simulare milioni di mosse per valutare l'albero delle possibilità. L'utilizzo di una mappa causava un grave overhead di memoria e tempi di accesso eccessivi che finivano per rallentare drasticamente le prestazioni e la profondità di ricerca del motore scacchistico.
- **Soluzione proposta (Array Monodimensionale e Metodi dedicati):** Per garantire performance ottimali, la struttura dati è stata sostituita con un Array monodimensionale (`Piece[]`). Inoltre, all'interno della `Board` sono stati introdotti metodi di accesso a bassissimo livello dedicati in via esclusiva al motore: `makeEngineMove`, `unmakeEngineMove` e `getPieceFast`. A differenza dei metodi standard, le prime due funzioni evitano di inquinare la `history` FEN, dato che le mosse simulate dalla CPU non necessitano di essere persistite nello storico. Infine, la funzione `getPieceFast` permette un accesso diretto all'array bypassando la creazione e l'allocazione in memoria di oggetti `Optional<Piece>`, un'operazione costosa se ripetuta milioni di volte al secondo. La sicurezza di questa chiamata è garantita a monte dai controlli preesistenti in `GameRules`.
- **Alternative scartate:** Mantenere la `Map<Position, Piece>` incapsulando i ritorni sempre all'interno di un `Optional<Piece>`. L'alternativa è stata abbandonata poiché il *Garbage Collector* entrava in azione troppo frequentemente a causa della massiccia allocazione di oggetti temporanei durante la fase di *thinking* della CPU.

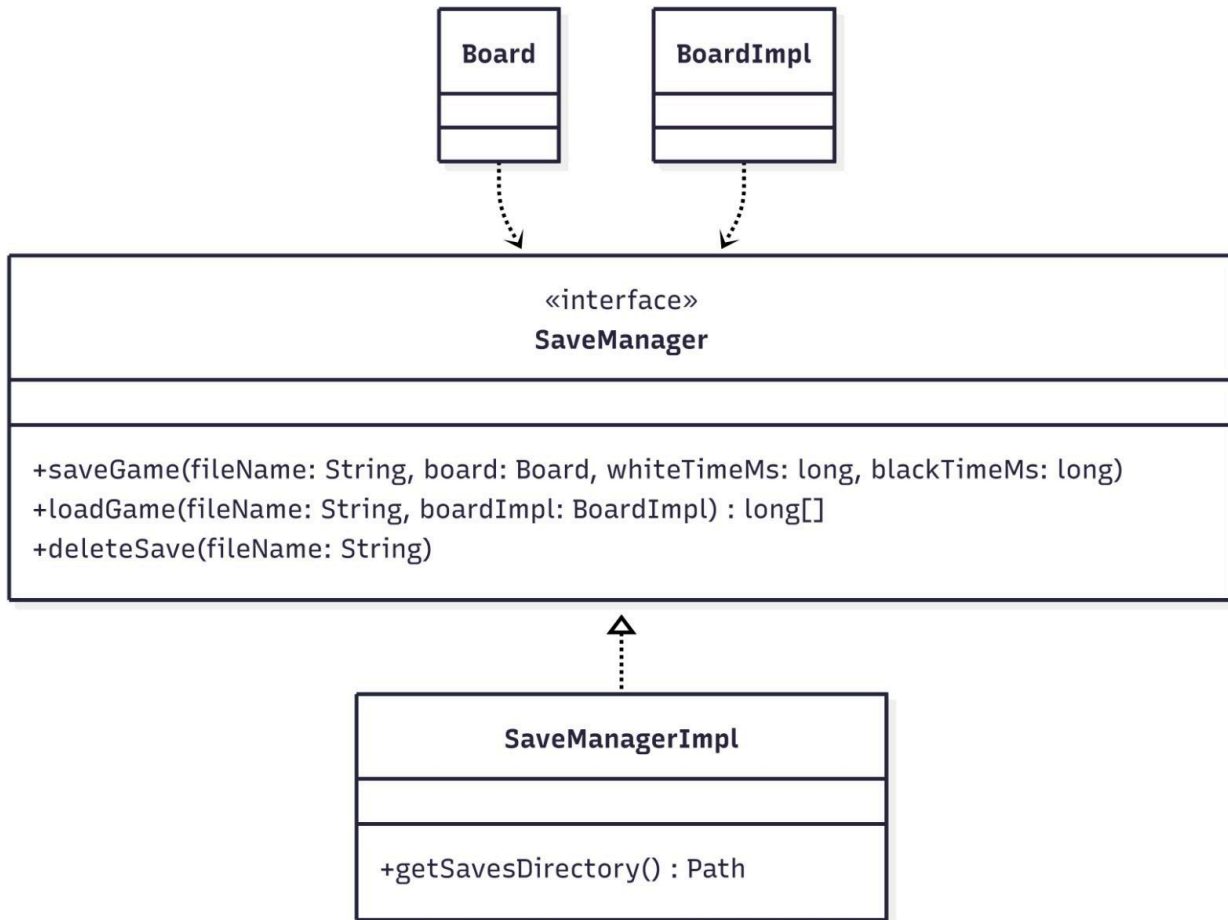
Disaccoppiamento della UI (Pattern Passive View)

- **Descrizione del problema:** In un'architettura MVC, l'interfaccia grafica deve occuparsi esclusivamente di visualizzare lo stato e catturare l'input dell'utente, senza possedere alcuna conoscenza delle regole del gioco. Se la View avesse invocato direttamente i metodi logici del Model o conosciuto la classe concreta del Controller per passargli i click, si sarebbe creato un accoppiamento bidirezionale letale, rendendo impossibile testare la logica isolatamente o sostituire l'interfaccia in futuro.
- **Soluzione proposta (Passive View e Callback):** La vista è stata ingegnerizzata seguendo il pattern Passive View tramite l'interfaccia astratta **ChessView**. Per gestire l'input senza conoscere chi lo elaborerà, la View espone metodi setter che accettano interfacce funzionali (come **Consumer<Position>** per i click sulle celle o **Runnable** per i bottoni). È il Controller che, all'avvio, inietta i propri comportamenti all'interno di questi listener. Quando l'utente clicca su una casella, **ChessViewImpl** si limita a invocare il **Consumer.accept(posizione)**, notificando l'evento in modo totalmente agnostico.
- **Alternative scartate:** Passare un riferimento diretto dell'oggetto **Controller** (la classe concreta) all'interno del costruttore di **ChessViewImpl**. Questa soluzione è stata scartata perché avrebbe violato il principio di inversione delle dipendenze: la GUI sarebbe diventata dipendente dalla specifica implementazione del controllo, rendendo il sistema rigido e difficile da mantenere.

Schema UML:



Schema UML:



Capitolo 3: Sviluppo

3.1 Testing automatizzato

La validazione del sistema è stata garantita tramite una robusta suite di test basata su **JUnit 5**:

- **AuraEngineTest**
 - Verifica che l'IA rispetti i limiti di tempo
 - Valuti correttamente i vantaggi materiali
 - Trovi matti in una mossa
 - Applichi correttamente arrocco e *en passant* quando ottimali.

Per testare metodi privati (es. `evaluateBoard`), è stata utilizzata la *Java Reflection API* tramite la classe di supporto `TestSupport`.

- **BoardTest:**
 - Assicura la corretta generazione delle stringhe FEN,
 - Il parsing protetto (eccezioni su FEN malformati)
 - Verifica che le operazioni di simulazione mossa e rollback ripristino esattamente lo stato originale.

- **GameRulesTest:**
 - Valida la legalità delle mosse
 - Controlli di scacco/matto/stallo
 - Le patte forzate (50 mosse, tripla ripetizione).
- **PieceMovementTest:**
 - Test dettagliati sul set di mosse specifiche
 - Prestando attenzione alle collisioni (pezzi amici/nemici) e ai bordi (fuori scacchiera).
- **SaveManagerTest:**
 - Verifica le operazioni di I/O (scrittura e lettura su file system), assicurandosi che file inesistenti sollevino le dovute `IOException` e che il caricamento ricostruisca correttamente la `history` per il rollback.

3.2 Note di Sviluppo

Federico Giorgetti

- **Utilizzo di Java Reflection API**
Utilizzata all'interno della suite di test (`AuraEngineTest`) per bypassare i modificatori di accesso e testare direttamente metodi privati cruciali come `evaluateBoard`, senza comprometterne l'incapsulamento nel codice sorgente.
Permalink:
<https://github.com/RiccardoDeMedio/OOP25-aurachess/blob/29220a87416aa2f23d05eab943d8ddff3e6000b/src/test/java/scacchi/model/ai/AuraEngineTest.java#L224>
- **Utilizzo dei Record**
Usati pervasivamente in `AuraEngine` per modellare i pacchetti di dati scambiati durante le computazioni (es. `Move`, `PlacedPiece`, `UndoInfo`). Questo garantisce l'immutabilità dei dati durante la discesa ricorsiva dell'algoritmo Minimax.
Permalink:
<https://github.com/RiccardoDeMedio/OOP25-aurachess/blob/29220a87416aa2f23d05eab943d8ddff3e6000b/src/main/java/scacchi/model/ai/AuraEngine.java#L513>

Michele Stanzani

- **Utilizzo di Optional**
Usato pervasivamente per modellare l'assenza di pezzi sulla scacchiera (es. metodo `getPieceAt(Position)`), forzando a tempo di compilazione la gestione dei casi vuoti ed eliminando il rischio di `NullPointerException` durante il calcolo delle mosse legali.
Permalink:
<https://github.com/RiccardoDeMedio/OOP25-aurachess/blob/29220a87416aa2f23d05eab943d8ddff3e6000b/src/main/java/scacchi/model/board/BoardImpl.java#L54>
- **Utilizzo del framework Java Collections (Set, HashSet)**
Impiegato estensivamente nelle classi dei pezzi (es. `AbstractSlidingPiece`, `AbstractSteppingPiece`) per collezionare e restituire le mosse legali in modo efficiente ed evitando duplicati.
Permalink:
<https://github.com/RiccardoDeMedio/OOP25-aurachess/blob/29220a87416aa2f23d05eab943d8ddff3e6000b/src/main/java/scacchi/model/pieces/AbstractSlidingPiece.java#L56>

Riccardo De Medio

- **Multithreading**

Utilizzato pervasivamente nel **Controller** e nella **View** per disaccoppiare i calcoli intensivi dell'Intelligenza Artificiale dal thread grafico (EDT), garantendo un'interfaccia sempre fluida e reattiva.

Permalink:

<https://github.com/RiccardoDeMedio/OOP25-aurachess/blob/29220a87416aa2f23d05eab943d8ddff-e3e6000b/src/main/java/scacchi/controller/Controller.java#L658>

- **Lambda Expressions e Interfacce Funzionali**

Usate pervasivamente nella View e nel Controller per la gestione dei Listener (es. **Consumer<Position>**, **Runnable**) e per l'esecuzione di task programmati tramite **javax.swing.Timer**.

Permalink:

<https://github.com/RiccardoDeMedio/OOP25-aurachess/blob/29220a87416aa2f23d05eab943d8ddff-e3e6000b/src/main/java/scacchi/view/ChessViewImpl.java#L154>

- **Utilizzo di Logger**

Utilizzato per il tracciamento degli eventi di sistema e dei tempi di esecuzione al posto delle classiche stampe a console.

Permalink:

<https://github.com/RiccardoDeMedio/OOP25-aurachess/blob/29220a87416aa2f23d05eab943d8ddff-e3e6000b/src/main/java/scacchi/app/AuraChess.java#L19>

Nicolò Paganini

- **Adozione delle API NIO.2**

Utilizzate per una gestione moderna, sicura e performante del file system all'interno della classe **SaveManagerImpl**, gestendo la creazione di directory nascoste e la lettura/scrittura dei file **.fen**.

Permalink:

<https://github.com/RiccardoDeMedio/OOP25-aurachess/blob/29220a87416aa2f23d05eab943d8ddff-e3e6000b/src/main/java/scacchi/model/savemanager/SaveManagerImpl.java#L44>

- **Utilizzo di ArrayDeque**

Scelta come implementazione preferenziale per lo storico mosse (**history**) all'interno di **BoardImpl**, garantendo efficienza massima nelle operazioni di stack (LIFO) richieste dalla logica di Rollback.

Permalink:

<https://github.com/RiccardoDeMedio/OOP25-aurachess/blob/29220a87416aa2f23d05eab943d8ddff-e3e6000b/src/main/java/scacchi/model/board/BoardImpl.java#L22>

- **Utilizzo di Espressioni Regolari**

Impiegate nella fase di I/O per sanitizzare i nomi dei file inseriti in input dall'utente, scartando i caratteri non sicuri e garantendo protezione contro vulnerabilità informatiche.

Permalink:

<https://github.com/RiccardoDeMedio/OOP25-aurachess/blob/29220a87416aa2f23d05eab943d8ddff-e3e6000b/src/main/java/scacchi/model/savemanager/SaveManagerImpl.java#L23>

Adozione di Git Hooks e SpotBugs: lo script **pre-commit** è ripreso integralmente da [OOP25-primus](#); il workflow **gradle-check.yml** e l'idea di collegare i controlli SpotBugs a un git hook pre-commit sono adattati dallo stesso progetto, poi integrati con il plugin **org.danilopianini.gradle-java-qa** fornito dal corso.

Permalink: [OOP25-primus](#)

Capitolo 4: Commenti finali

4.1 Autovalutazione e lavori futuri

Federico Giorgetti

Il lavoro di sviluppo su AuraEngine si è focalizzato sulla realizzazione di un sistema decisionale capace di gestire l'elevata complessità computazionale del gioco degli scacchi. Sono particolarmente soddisfatto dell'integrazione dell'algoritmo Alpha-Beta pruning, che ha permesso di ottimizzare significativamente l'esplorazione dell'albero delle mosse, riducendo lo spazio di ricerca e migliorando le performance in tempo reale. Parallelamente, l'implementazione dell'Aurometro ha fornito un riscontro quantitativo coerente, traducendo la valutazione tattica della posizione in un parametro di facile interpretazione per l'utente.

Analizzando criticamente l'architettura attuale, riconosco margini di miglioramento legati alla scalabilità dell'algoritmo. Nonostante l'efficienza raggiunta, il motore potrebbe beneficiare di ulteriori ottimizzazioni euristiche: in particolare, l'adozione di tecniche di move ordering avanzate come l'ordinamento prioritario delle catture o, l'impiego di tabelle di trasposizione basate su Zobrist Hashing, permetterebbe di ridurre drasticamente le collisioni e i ricalcoli. Inoltre, la precisione valutativa dell'Engine potrebbe essere ulteriormente raffinata integrando la notazione ufficiale in Centipawn, passando così da una stima relativa a una scala standardizzata di riferimento per la teoria scacchistica.

Michele Stanzani

Il mio contributo principale al progetto si è concentrato sulla modellazione del dominio fisico degli scacchi, occupandomi in prima persona dell'architettura della gerarchia dei pezzi e della complessa gestione dei loro movimenti. Fin dalle prime fasi, l'obiettivo è stato evitare una classe monolitica di validazione che avrebbe reso il codice rigido e difficile da mantenere.

Ritengo che l'adozione di un design polimorfico, unito all'impiego del pattern Template Method, sia stato un enorme successo in termini di ingegneria del software. Attraverso l'introduzione di classi astratte intermedie come `AbstractSlidingPiece` e `AbstractSteppingPiece`, è stato possibile centralizzare interamente la logica di iterazione spaziale e il rilevamento delle collisioni. Grazie a questo livello di astrazione, i singoli pezzi concreti devono limitarsi a fornire i propri vettori direzionali, riducendo di fatto la duplicazione del codice a zero e aderendo in modo rigoroso al principio Open/Closed. Ad arricchire questa infrastruttura si è aggiunto l'uso del pattern Factory Method (`PieceFactory`), che ha permesso di disaccoppiare l'istanziamento dei pezzi dal parsing delle stringhe FEN.

La parte tecnicamente più sfidante del mio lavoro è stata senza dubbio l'isolamento delle mosse speciali del Pedone. A causa della sua forte asimmetria intrinseca si muove solo frontalmente ma cattura in diagonale, oltre a possedere la meccanica unica del doppio passo iniziale ho dovuto studiare una soluzione che non forzasse il suo comportamento all'interno di astrazioni pensate per pezzi simmetrici, portandolo a implementare direttamente l'interfaccia `Piece`.

Guardando agli sviluppi futuri, la solida infrastruttura orientata agli oggetti che è stata costruita si presta perfettamente a ulteriori espansioni. Sarebbe infatti estremamente interessante astrarre e parametrizzare ulteriormente le logiche vettoriali dei movimenti: questo permetterebbe la facile introduzione di pezzi inventati e non convenzionali, aprendo la strada all'implementazione di varianti esotiche degli scacchi senza dover minimamente intaccare o riscrivere le fondamenta del motore di base.

Riccardo De Medio

Il mio ruolo all'interno del progetto si è focalizzato sul compito di mediatore, occupandomi di legare il backend algoritmico con l'interazione umana attraverso lo sviluppo del Controller, e gestendo al contempo le regole più complesse e sfaccettate del dominio scacchistico.

Sono particolarmente fiero delle scelte architetture adottate per la validazione delle mosse. Aver ingegnerizzato la classe `GameRules` mantenendola "pura" e rigorosamente senza stato (in piena aderenza al pattern Pure Fabrication) ha rappresentato un successo cruciale. Questo approccio, unito all'uso del Decorator `SimulatedBoard`, ha permesso di prevedere scenari complessi, come l'auto-scacco o le limitazioni sull'arrocco, in totale sicurezza, proteggendo la scacchiera reale da corruzioni accidentali e mantenendo il codice altamente coeso e testabile.

Un altro aspetto di cui vado orgoglioso è l'orchestrazione del flusso di gioco e dei turni tra giocatore umano e Intelligenza Artificiale. L'integrazione del calcolo asincrono tramite `SwingWorker` ha permesso di delegare le pesanti simulazioni dell'engine a un thread secondario, garantendo che l'interfaccia grafica non si "freezasse" mai e mantenendo intatta la fluidità dell'esperienza utente. A questo si aggiunge l'implementazione del report di fine partita: incrociando lo storico del giocatore con l'analisi matematica dell'Aurometro, il sistema genera un riepilogo dettagliato mossa per mossa, conferendo all'applicativo un reale e tangibile valore didattico.

Per quanto riguarda gli sviluppi futuri, mi piacerebbe sostituire l'attuale sistema di puntamento a doppio click con il supporto nativo al drag-and-drop: questo renderebbe l'interazione con la scacchiera molto più moderna, tattile e naturale, sebbene richieda una gestione decisamente più complessa degli eventi del mouse e del rendering grafico in tempo reale.

Nicolò Paganini

Lo sviluppo di AuraChess ha rappresentato per me una sfida tecnica notevole e un'importante occasione di crescita professionale. Essendo una delle mie prime esperienze con un progetto software così strutturato e con l'uso di strumenti di versionamento cooperativo come Git, l'impatto iniziale è stato impegnativo, ma ha consolidato in me competenze architetture fondamentali.

Il mio dominio di competenza si è rivelato ampio e trasversale. Da un lato, mi sono occupato della gestione della scacchiera e della persistenza dei dati. L'intuizione di usare le stringhe FEN come reificazione del pattern Memento per la Board si è rivelata vincente: ha reso banalmente semplice implementare il Rollback, salvare la partita su file di testo leggerissimi e verificare le patte per ripetizione, tutto con il medesimo meccanismo.

Dall'altro lato, ho preso in carico quasi l'intera implementazione dell'interfaccia grafica (`ChessView` e `ChessViewImpl`), l'entry point dell'applicazione (`AuraChess`) e la delicata questione del multithreading. Progettare una View rigorosamente passiva e disaccoppiata dal Model ha richiesto un grande sforzo di astrazione. La sfida più complessa è stata senza dubbio la sincronizzazione dei thread: implementare lo `SwingWorker` per garantire un'interfaccia fluida e reattiva durante i calcoli intensivi dell'Intelligenza Artificiale mi ha fatto scontrare con i limiti dell'Event Dispatch Thread di Java Swing. Risolvere problemi legati a potenziali race conditions e freeze dell'applicazione mi ha spinto a esplorare soluzioni di design avanzate, elevando nettamente la qualità del mio codice.

Come nota sul processo di sviluppo, concordo sul fatto che far dialogare componenti così strettamente disaccoppiate (come la mia interfaccia grafica e il motore logico gestito dai colleghi) richieda una pianificazione impeccabile, spesso difficile da mantenere in un gruppo giovane e sotto le scadenze degli appelli. Un maggiore indirizzamento pratico nelle fasi iniziali del corso, specialmente su come tradurre un'architettura complessa in una relazione tecnica così formale, avrebbe forse aiutato a ottimizzare i tempi e a ridurre il disorientamento durante la stesura documentale.

Guardando al futuro, considerando la solida base architetture raggiunta, vorrei implementare l'esportazione e l'importazione delle partite nel formato standard PGN (Portable Game Notation). Questo non solo arricchirebbe le funzionalità del gioco, ma garantirebbe la totale compatibilità con database scacchistici esterni, rendendo AuraChess uno strumento ancora più completo per l'analisi.

Guida utente

All'avvio dell'applicazione verrai accolto da un Menù iniziale tramite pop-up, con il seguente caloroso benvenuto "Benvenuto in AuraChess! ".

Avvio del gioco Potrai scegliere tra tre opzioni:

1. **Nuova Partita:** Inizia una partita classica (tu giochi con il Bianco, la CPU controlla il Nero).
2. **Carica Vecchia Partita:** Apre un menù a tendina con i salvataggi presenti sul tuo PC, permettendoti di riprendere esattamente da dove avevi lasciato.
3. **Gestisci Salvataggi:** Un pannello di amministrazione per eliminare salvataggi specifici o ripulire l'intero storico dal tuo computer.

Come giocare

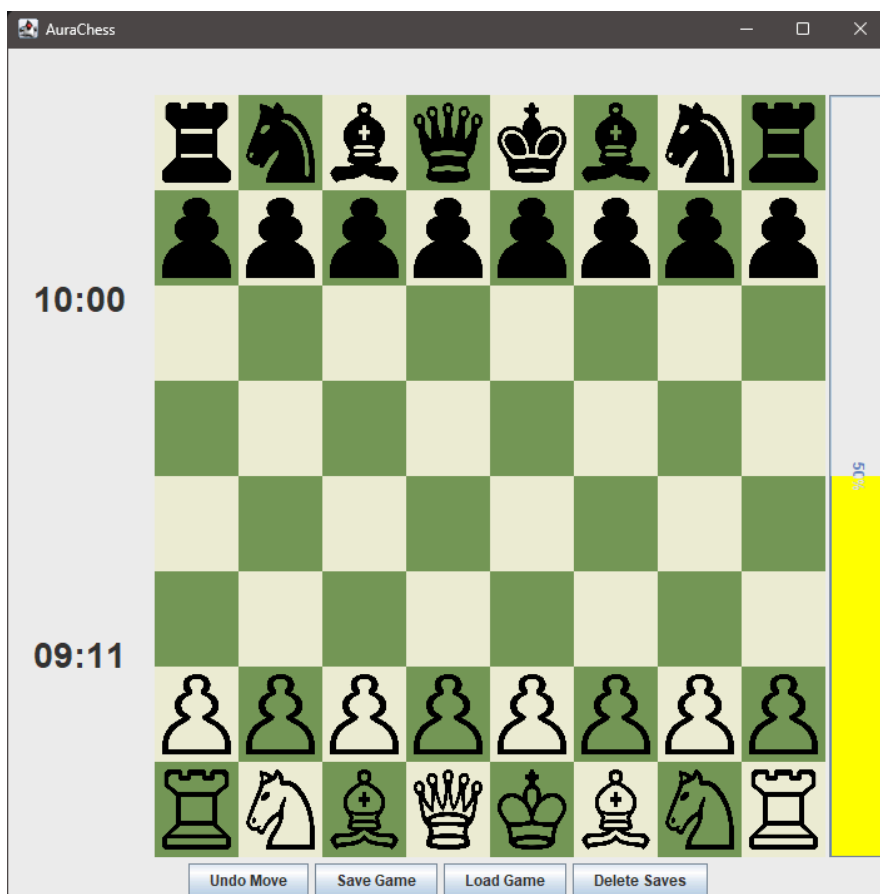
- **Selezione e Movimento:** Clicca sul pezzo che desideri muovere. La casella si evidenzierà. Clicca sulla casella di destinazione per confermare la mossa. Se cambi idea, clicca di nuovo sul pezzo per deselezionarlo.
- **Aurometro:** Sulla destra della scacchiera c'è una barra verticale colorata. Questo è l'Aurometro: calcola in tempo reale la precisione delle tue giocate
- **Valutazione:** Immediatamente dopo una tua mossa, apparirà un pop-up in alto (es. "Eccellente", "Imprecisione", "Mossa pessima") per aiutarti a migliorare.
- **Pulsantiera inferiore:**
 - **Undo Move:** Hai commesso un errore? Clicca qui per annullare la tua ultima mossa e quella della CPU, riportando il gioco allo stato precedente.
 - **Save Game:** Salva la partita in corso. Ti verrà chiesto di inserire un nome per il salvataggio.
 - **Load Game / Delete Saves:** Puoi gestire i file anche durante una partita.

Fine della Partita La partita termina automaticamente in caso di Scacco Matto, Patta o Tempo Scaduto. Al termine, il sistema genererà un **Report di Analisi** completo che confronterà, mossa per mossa, le tue scelte con quelle ottimali suggerite dal motore scacchistico.



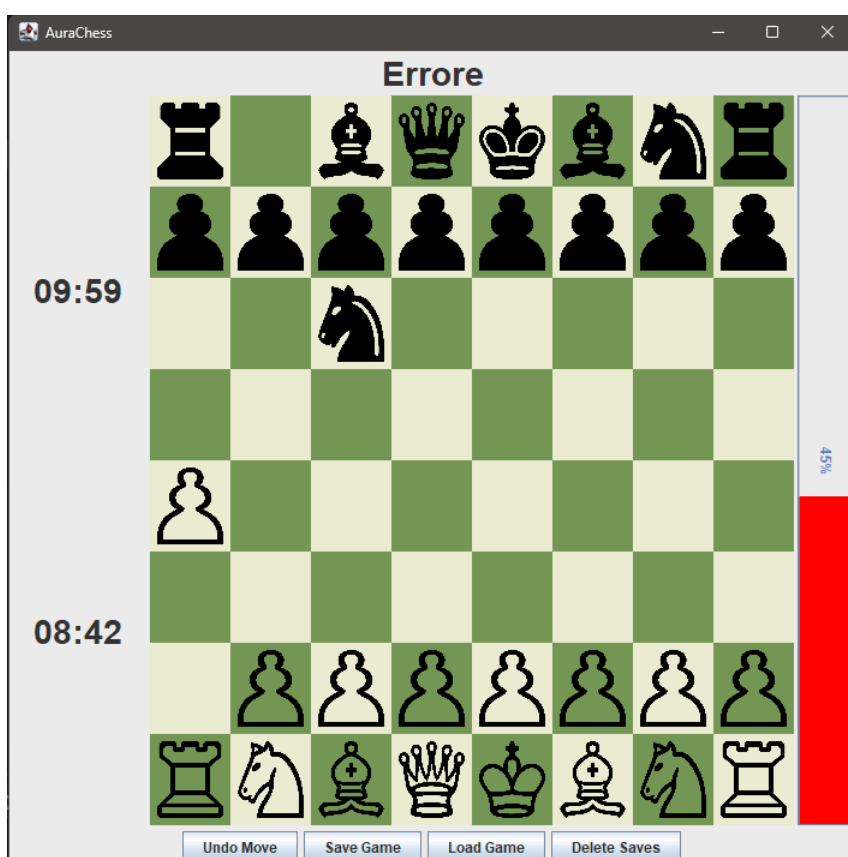
Menu di avvio di AuraChess:

Può essere creata una nuova partita, ripresa una partita già iniziata o eliminare i salvataggi precedenti.



Scacchiera:

In basso i pezzi bianchi, in alto i pezzi neri. Sul lato sinistro c'è l'orologio con il timer indicante il tempo rimanente. In basso vi sono le opzioni per salvare e per tornare indietro annullando le mosse. A destra è presente "l'Aurometro", il quale determina l'andamento della partita. Di default il parametro dell'Aurometro è impostato sul cinquanta per cento.

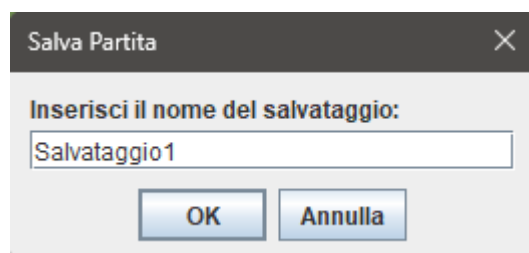


Mossa sbagliata effettuata:

In alto è presente il messaggio con l'esito della mossa appena giocata, e a destra il punteggio diminuito dopo la mossa imprecisa.

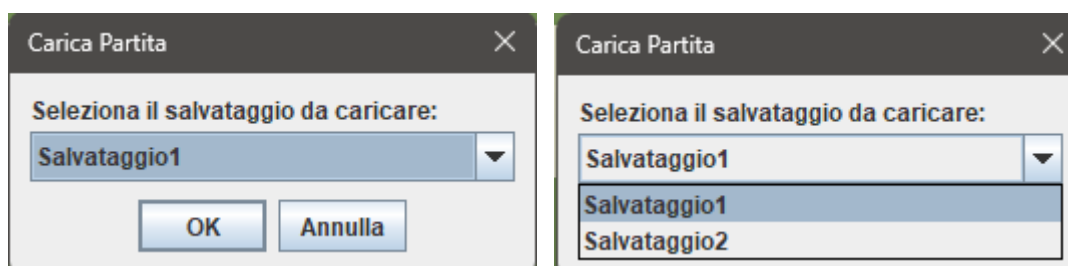
Salvataggio della partita:

Funzione di salvataggio della partita con un nome che si può scegliere, in questo caso il file viene chiamato come “Salvataggio1”.



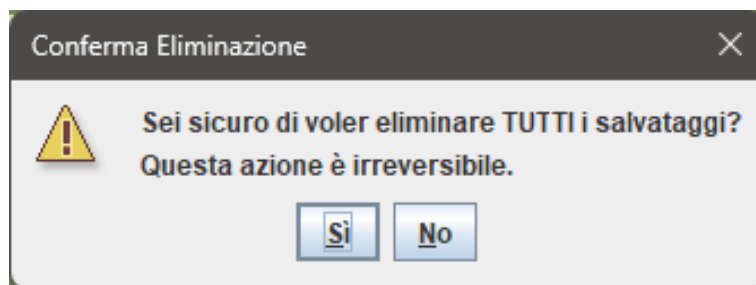
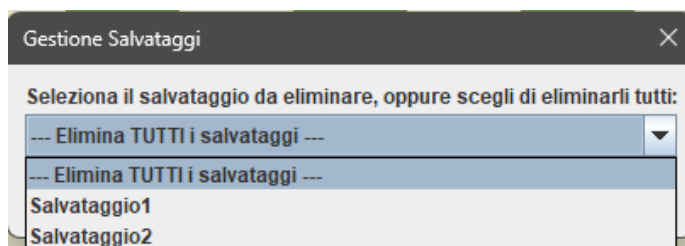
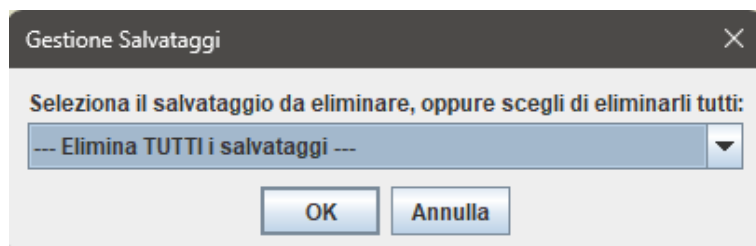
Gestione Caricamento dei salvataggi:

Si apre un pop-up a tendina che permette di selezionare tra tutti i file salvati quale partita caricare.



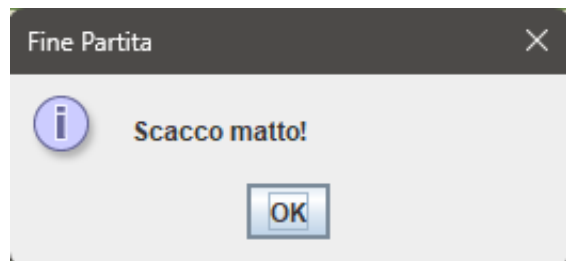
Gestione eliminazione dei salvataggi:

Si apre un pop-up a tendina, permettendo la selezione di quale salvataggio eliminare, con inoltre il comando che permette l'eliminazione di tutti i salvataggi, sia in caso di cancellazione di singolo salvataggio, che di tutti i salvataggi viene richiesta conferma per l'eliminazione del file.



Avviso di fine partita:

A seconda di come finisce la partita, compare il messaggio di come è finita la partita, Scacco Matto, Timer ecc...



Analisi Partita:

A fine partita appare l'analisi della partita, con numero del turno, seguito da mossa eseguita con la valutazione della mossa e affianco la mossa migliore che si poteva eseguire.

